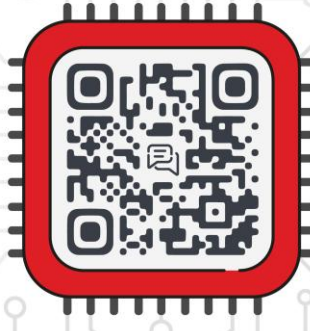




9



14



1680 sp

عملي

كلية الهندسة المعلوماتية

السنة الثانية

B-Tree

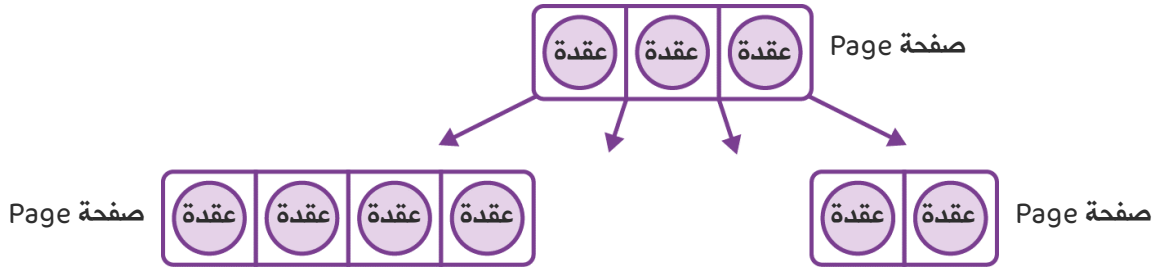


د. عملي مشترك

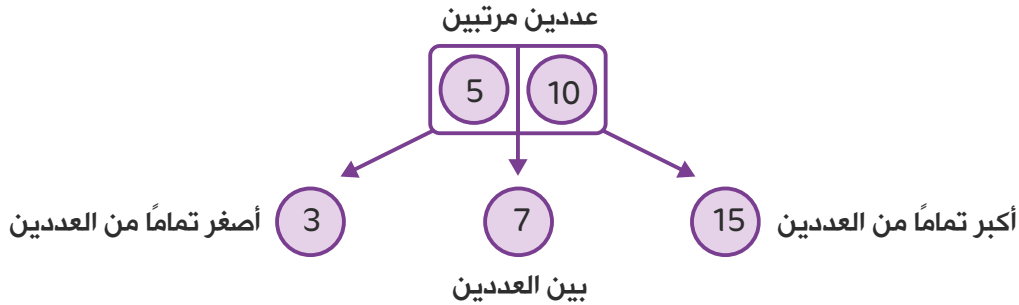
RB Informatics; 14/06/2025

خوارزميات ٢

تتلخص الفكرة بتقسيم شجرة البحث الثنائية إلى أشجار جزئية وصفحات "Pages" بحيث يتم التعامل مع كل صفحة كوحدة متكاملة تنسخ مرة واحدة إلى الـ RAM دون الحاجة لكلفة إضافية، وهكذا يمكن تمثيل الصفحة بمصفوفة أو Linked List.



مثال:



للتحكم بطريقة كبر الشجرة المعممة (توزيع الصفحات والأبناء) نتبع شروط الـ B-Tree التي وضعها العالم Bayer.

شروط الـ B-Tree

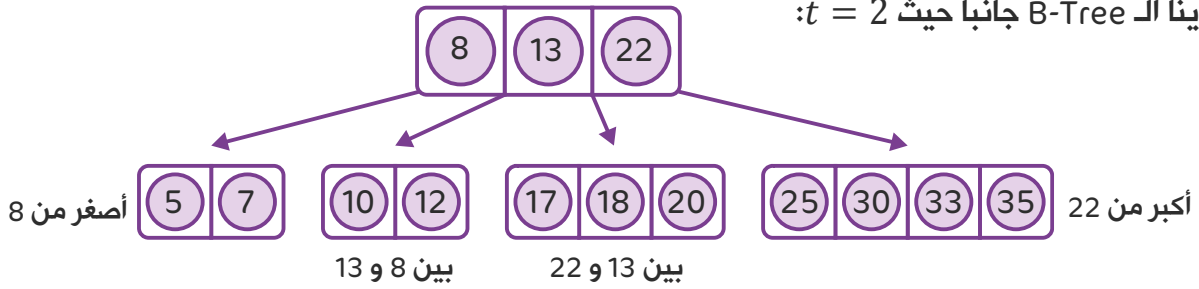
1. يجب أن تحوي كل صفحة على t عقدة على الأقل، و $2t$ على الأكثر. حيث t : رتبة الشجرة ونحن من نحددها مسبقاً، **فمثلاً** ($t = 2$) أي كل Page فيها عقدتين أو ثلاث أو أربع.

يوجد استثناء للـ Root: ليس لعدد العقد حد أدنى ولكن الحد الأعلى موجود، أي عدد العقد ($1 \rightarrow 2t$).

2. لكي أبحث يجب أن تكون عقد كل صفحة مرتبة.
3. يجب أن تكون جميع أوراق الشجرة بنفس المستوى وهو آخر مستوى.
4. إذا كان لدينا صفحة فيها m عقدة فإما أن لا يكون لها أبناء، لها $(m + 1)$ ابن.

بعد تحقيق هذه الشروط أصبح يمكننا البحث فيها بتعقيد أقل من $\log_2 n$.

مثال: لدينا الـ B-Tree جانباً حيث $t = 2$:



نلاحظ أن الشجرة أعلاه تحقق جميع الشروط الأربعة.

البحث

ستكون آلية البحث على مرحلتين:

1. الانتقال للصفحة التالية:

2. البحث ضمن الصفحة الواحدة:

طالما لم نجد العنصر فهو:

- إما أكبر من كل العناصر.
- إما أصغر من كل العناصر.
- أو يقع بين عنصرين.

وحسب حالة العنصر من الحالات السابقة نقرر الصفحة التي سنذهب إليها (بكل مرة ندخل سهم واحد فقط).

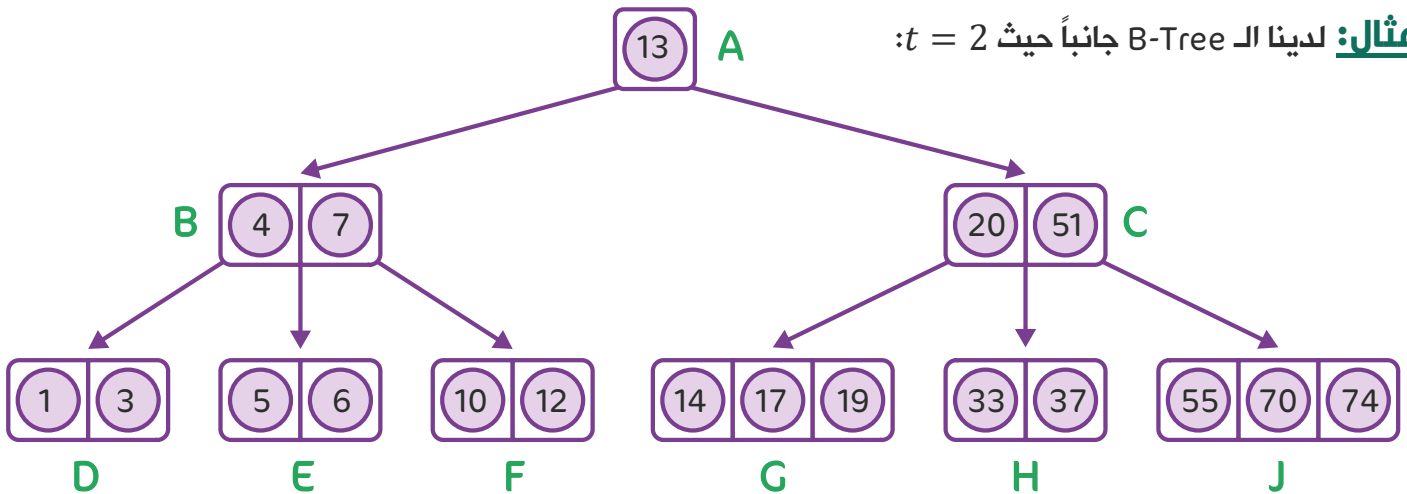
نقارن العدد المطلوب بعناصر الصفحة



وفي حال لم نجد العنصر المطلوب ننتقل للصفحة التالية.

← **إذا:** في كل مرة نبدأ من صفحة الـ Root ونبحث، في حال عدم وجود العنصر ننتقل للصفحة التالية وهكذا، نتوقف عند إيجاد العنصر أو عند الوصول لورقة (إذاً نتوقف حسب ارتفاع الشجرة).

مثال: لدينا الـ B-Tree جانباً حيث $t = 2$:



1. سنبحث عن 8:

- نبحث في صفحة الـ Root: لم نجد 8، و 8 أصغر من 13 إذاً سنتوجه لليمين (الصفحة B).
- نبحث في الصفحة B: لم نجد 8، و 8 أكبر من عناصر الصفحة إذاً سنتوجه لليسار (الصفحة F).
- نبحث في F: لم نجد 8 والصفحة F ورقة
- ← 8 عنصر غير موجود بالشجرة.

2. سنبحث عن 33:

- نبحث في صفحة الـ Root: لم نجد 33، و 33 أكبر من 13 إذاً سنتوجه لليمين (الصفحة C).
- نبحث في الصفحة C: لم نجد 33، و هي محصورة بين 20 و 51 إذاً نتوجه للسهم بينهما (الصفحة H).
- نبحث في H: فنجد العقدة 33.

الإضافة

تتم عملية الإضافة في الـ B-Tree بسهولة حيث تكون بالأوراق فقط وتتبع الخطوات التالية:

1. البحث عن العنصر الذي نريد إضافته:

يؤدي البحث عن العنصر الذي نريد إضافته إلى تحديد مكان (صفحة) الإضافة.

2. إضافة العنصر إلى الصفحة التي حددناها، وهنا نميز حالتين:

الصفحة التي نريد الإضافة إليها ممتلئة

تحتوي $2t$ عنصراً:

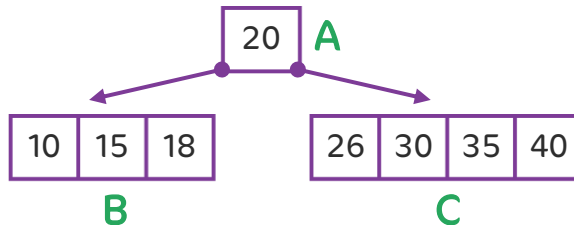
- فإذا أضفنا العنصر سيصبح عدد عناصر الصفحة $(2t + 1)$ ولحل هذه المشكلة:
- 1- نأخذ أول t عنصر ونضعه بصفحة.
 - 2- نأخذ آخر t عنصر ونضعه بصفحة.
 - 3- العنصر في المنتصف نضيفه لصفحة الأب.

الصفحة التي نريد الإضافة إليها تحوي عناصر

بعدد بين t و $2t$ وأصغر تماماً من $2t$:

عندها نضيف العقدة دون إجراء أي معالجة.

مثال: لدينا الـ B-Tree التالية حيث $t = 2$:



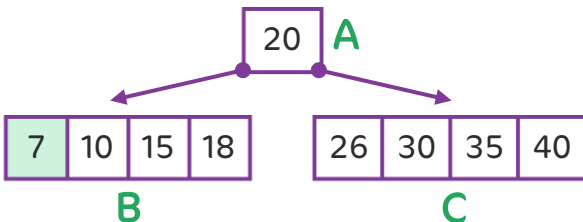
■ سنقوم بإضافة 7:

■ نبحث عن الـ 7 ستوصلنا للصفحة B وبعدها ضمن

الصفحة سنجد أن الـ 7 أصغر من كل العناصر

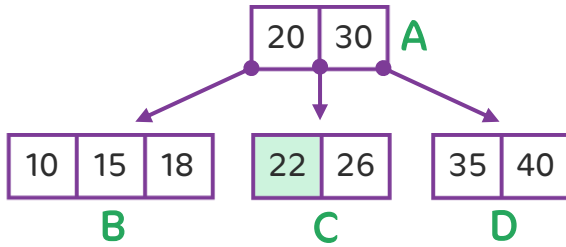
وإذا قمنا أيضاً بـ Binary Search سنصل لنفس المكان

فببساطة نضيف الـ 7:



■ سنقوم بإضافة 22:

C 22 26 30 35 40



■ بعد عملية البحث سنصل للصفحة C،

ونجد أنها ممتلئة، فنضيف الـ 22 في المكان المناسب لها.

■ أصبح لدينا $(2t + 1)$ عنصر، لذا نأخذ أول t عنصر من اليمين نضعهم بصفحة (D) وأول t عنصر من اليسار نضعهم بصفحة (C) وستكون هذه الصفحات إخوة، والعنصر في المنتصف سيعود للأب ونضعه بجهة فرع الصفحة المفككة، وتصبح الشجرة كالموجودة جانباً:

ملاحظة: في المثال السابق لا يخل الشرط الرابع لأن الصفحة A زاد عدد عناصرها بمقدار عنصر واحد وزاد عدد أبنائها (الصفحات) بمقدار واحد.

عندما نرفع العقدة إلى الأب سنميز حالتين:

الصفحة الأب تحوي $2t$:

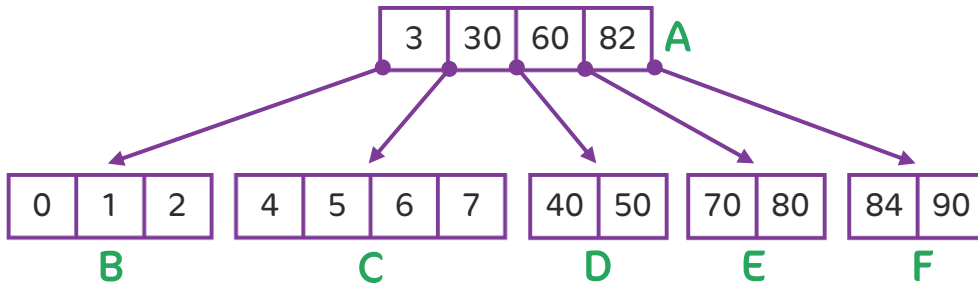
إذا لا يوجد مكان متاح ← نكرر عملية الإضافة عند الصفحة الحالية (الخاصة بالأب) وهكذا نكرر العملية حتى الوصول لصفحة غير ممتلئة (من أحد الآباء) نضيف عليها.

الصفحة الأب تحوي أقل من $2t$:

إذا يوجد مكان متاح ← سنضيف العنصر في مكانه كما في الحالة السابقة.

ملاحظة: إذا وصلنا لا Root وكان ممتلئاً نقوم بنفس العملية، ولإضافة عنصر للأب نقوم بفتح مستوى جديد فوق الـ Root أي يصبح لدينا Root جديد، ولهذا يوجد إستثناء لا Root بأن يحمل عنصر واحد.

مثال: لدينا الـ B-Tree جانباً حيث $t = 2$:



■ سنقوم بإضافة 8:

بعد البحث نجد المكان المناسب هو الصفحة C،

وهي ممتلئة وبعد الإضافة سيصبح عدد العناصر $(2t + 1)$ إذاً اختلت الـ B-Tree.

■ نقسم الصفحة (C) من اليمين واليسار كما تعلمنا:

أصبح المستوى الأخير كالتالي:

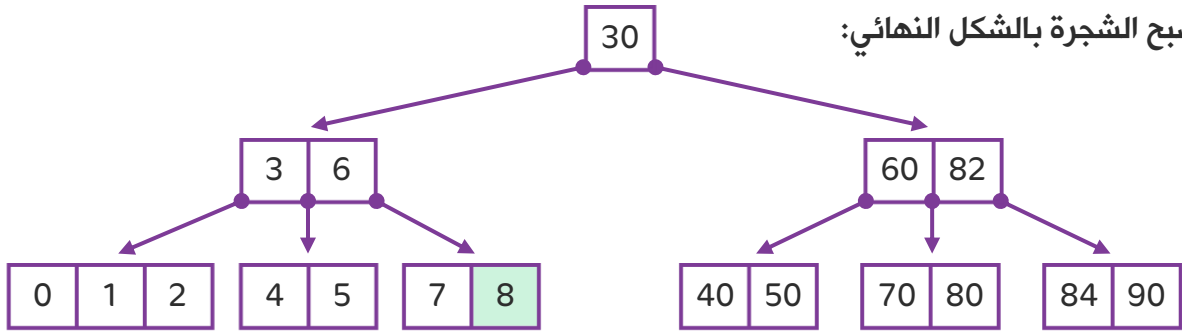


A 3 6 30 60 82

3 6 60 82

30

ال Root الجديد



والعنصر 6 الذي يقع في المنتصف سيعود
لصفحة الأب (A) والتي هي ممتلئة أيضاً:

- نقسم الصفحة (A) من اليمين واليسار كما تعلمنا:
والعنصر 30 الذي يقع في المنتصف سيعود،
وبما أننا أصلاً في ال Root سيصبح لدينا
العنصر 30 هو ال Root الجديد.
- تصبح الشجرة بالشكل النهائي:

ملاحظة: عندما قسمنا ال A قسمين فإننا نعيد ترتيب الأبناء كما كانت، مثلاً B كانت أصغر ما يمكن (أقصى اليسار)، و C بين أول عنصرين وهكذا.

تعقيد عملية الإضافة:

1. كلفة الإضافة إلى الصفحة:
 $O(t)$ في المصفوفة وال- Linked List (لأنه في ال- Linked List نحتاج للبحث عن العنصر).
 2. كلفة الإصلاح:
كما رأينا بأسوأ الأحوال سنضطر لإصلاح جميع الصفحات حتى نصل للجذر، إذاً سنقوم بـ h إصلاح أو $\log_t n$ إصلاح وبعدها تعقيد الإضافة للصفحة كما سبق.
- ← أصبح التعقيد: $O(t \cdot \log_t n)$.

ملاحظات:

- نحسب أيضاً تعقيد عملية البحث عن مكان الإضافة وهو $O(\log_2 t \cdot \log_t n)$ ، رغم ذلك يبقى التعقيد كما هو تقريباً (من نفس الرتبة):

$$O(\log_2 t \cdot \log_t n + t \cdot \log_t n) \in O(t \cdot \log_t n)$$

- في حال اعتبرنا t عدد ثابت تصبح الإضافة بتعقيد $O(\log_t n)$ وهو نفسه $O(h)$ أي يمكننا القول أنه حافظنا على عملية الإضافة بتعقيد $O(h)$ تقريباً.

الحذف

نميز حالتين

الحذف من صفحة داخلية.الحذف من ورقة.

1. الحذف من ورقة:

أيضاً نميز حالتين

الصفحة فيها t عقدة:

وفي حال حذف العقدة سيصبح عدد العقد في الصفحة

أقل من t ، إذاً ستختل B-Tree، **فما الحل؟؟**الصفحة فيها أكثر من t عقدة:

نطبق عملية الحذف بكل بساطة

في الحالة الثانية (الصفحة فيها t عقدة):

نقوم باستعارة عقدة من الأب، هكذا يصبح عدد العقد في الصفحة الابن t عقدة، وهنا نواجه مشكلة أخرى، إذ نعلم أن كل صفحة تحوي m عقدة لها $m + 1$ ابن وعند استعارة عقدة من الأب ستقل m بمقدار واحد وعدد الأبناء سيبقى $m + 1$ ، ولحل هذه المشكلة نميز حالتين:

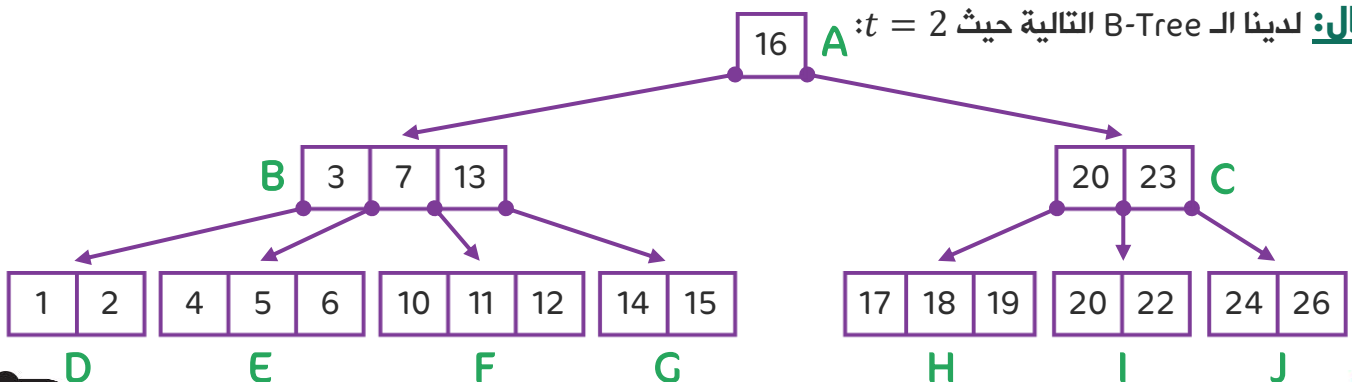
إذا كان للصفحة الابن أخ يحوي t عقدة:

ندمج الصفحة الابن مع الأخ، فالابن فيها t عقدة والأخ فيها t عقدة إذاً الصفحة الجديدة ستحوي $2t$ عقدة ولن تختل الشجرة.

إذا كان للصفحة الابن أخ يحوي أكثرمن t عقدة:

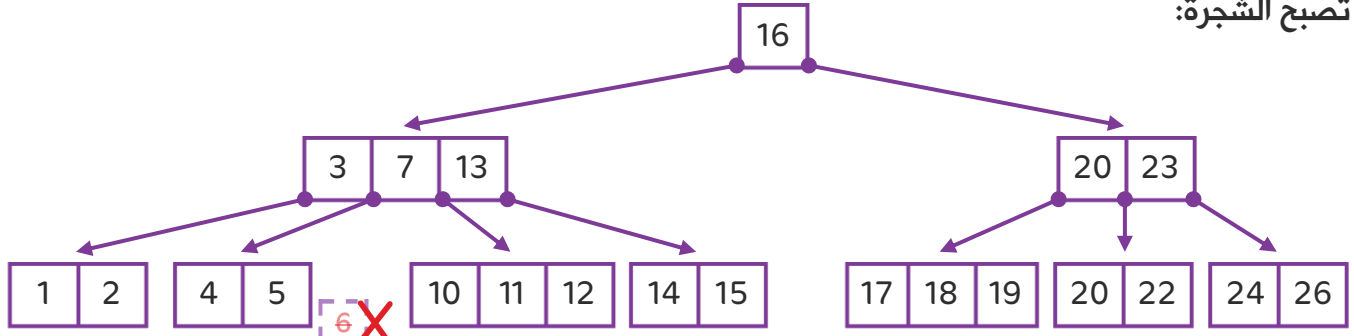
سيعوض الأخ الأب عن العقدة التي فقدها.

وهكذا نكون قد حذفنا من ورقة مع الحفاظ على بنية الـ B-Tree.

مثال:لدينا الـ B-Tree التالية حيث $t = 2$ A:

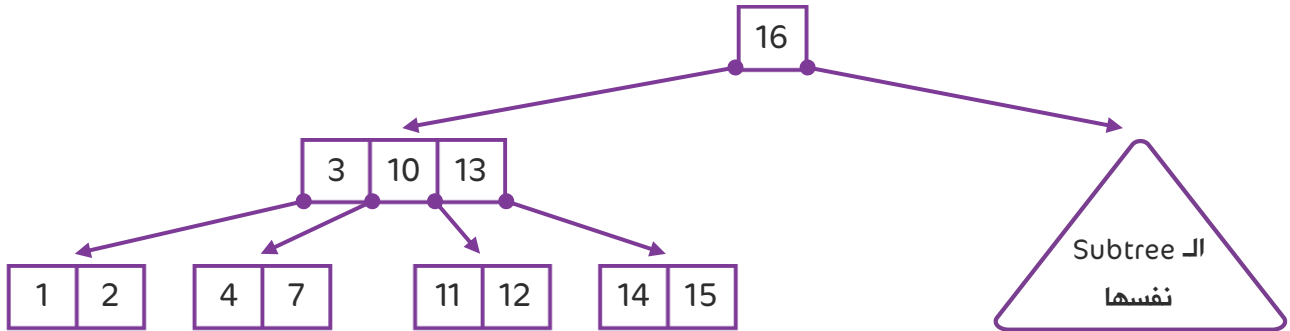
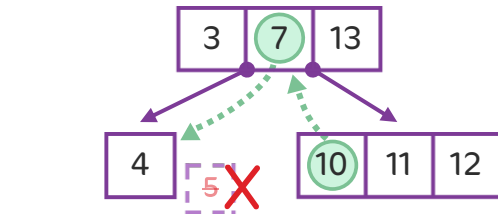
حذف العقدة 6:

نفكر كما يلي: العقدة 6 موجودة في الصفحة E وهي ورقة وتحتوي أكثر من t عقدة ← نحذف العقدة ببساطة بدون تغيير.
← تصبح الشجرة:



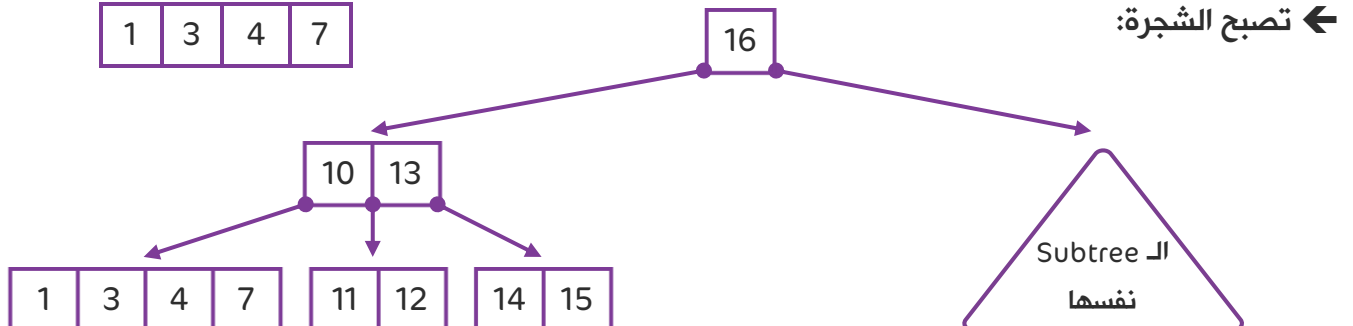
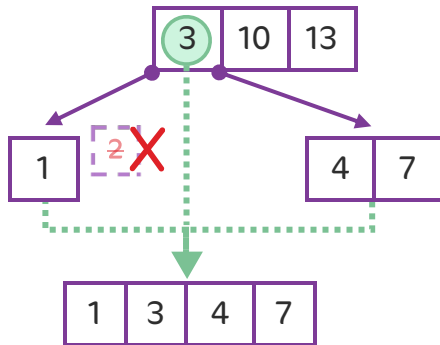
حذف العقدة 5:

العقدة 5 موجودة في الصفحة E وهي ورقة وتحتوي t عقدة
← أخ الصفحة E هو F يحتوي أكثر من t عقدة فيمكن
التعويض عن طريق الأخ، فنضع مكان الـ 5 عقدة من الأب
وسيقوم الأخ بتعويض الأب عن العقدة المفقودة:
← تصبح الشجرة:



حذف العقدة 2:

العقدة 2 موجودة في الصفحة D وهي ورقة وتحتوي t عقدة
← لا تملك D أخ لديه أكثر من t عقدة، سنحذف 2 وندمج
العقدة 1 المتبقية مع الأخ E بالإضافة لعقدة ننزلها من الأب:



2. الحذف من صفحة داخلية:

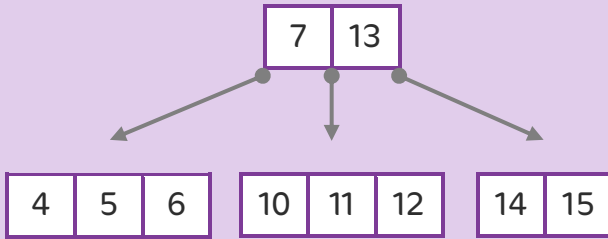
وهنا سنميز أربع حالات مرتبة بحيث إذا لم تتحقق الأولى نلجأ للثانية وإذا لم تتحقق الثانية نلجأ للثالثة وهكذا.

تذكرة في العنصر التالي والعنصر السابق:

من الشجرة جانباً نجد أن:

- العنصر التالي للعقدة 13 هو 14.

- العنصر السابق للعقدة 13 هو 12.



↪ نعود لحالات الحذف الأربع:

العنصر الذي نريد حذفه يملك عنصر التالي أوسابق أحدهما قابل للحذف البسيط.

1

مثلاً في الشجرة السابقة في التذكرة إذا حذفنا

12 لن تختل صفحته على عكس 14 إذا قمنا

بحذفه ستختل صفحته.

الصفحتين التالية والسابقة للعقدةالمراد حذفها (ابنا العقدة)يملكان t عقدة "الحد الأدنى".

2

صفحة العقدة التي نريدحذفها لها ابن يملك عقدأكثر من الحد الأدنى t .

3

هذه الحالة هي نفي جميعالحالات السابقة:

4

وهي تمثل حالة جميع أبناء وأحفاد صفحة

العقدة المراد حذفها تحوي t عقدة فقط.

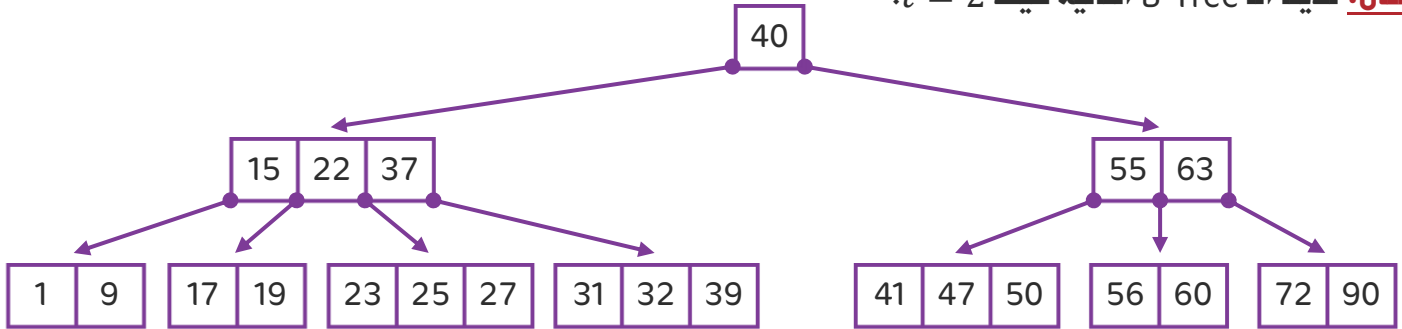
ملاحظات:

- سنرى آلية التعامل مع كل حالة لكن الهدف من عرضهم هكذا بالبداية هو بناء لمخطط لكل حالة.
- في بعض الأحيان يكون بإمكاننا معالجة الشجرة باستخدام الطريقة الثانية أو الثالثة (نكون مخيرين بينهما) لكن قد نجبر على أحدهما أيضاً.

1. الحالة الأولى: العنصر الذي نريد حذفه يملك عنصر تالي أو سابق أحدهما قابل للحذف البسيط:

نفحص إمكانية حذف العنصر السابق ثم إمكانية حذف العنصر التالي ثم نبدل الممكن منهم مع العنصر المراد حذفه وببساطة يتم حذف العنصر.

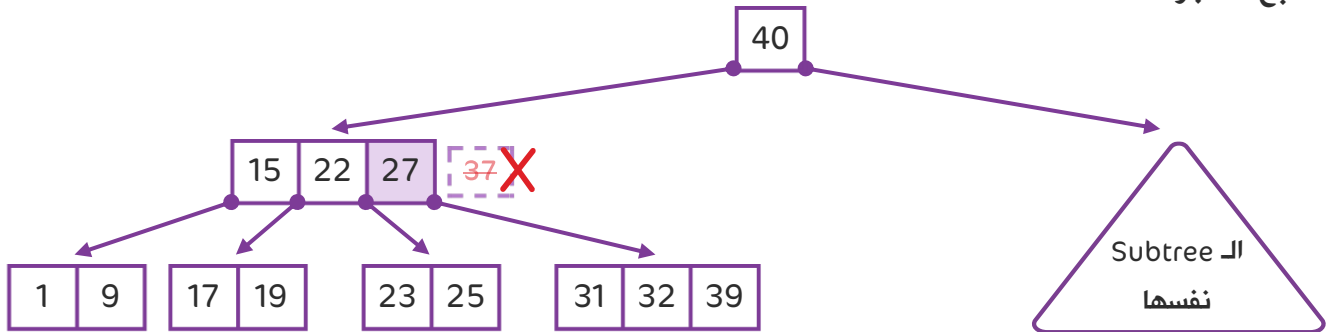
مثال: لدينا الـ B-Tree التالية حيث $t = 2$:



▪ حذف العقدة 37:

العقدة موجودة بصفحة داخلية ولها عنصر سابق قابل للحذف البسيط هو 27 ← نبدل 27 مع 37 ونحذف 37.

← تصبح الشجرة:

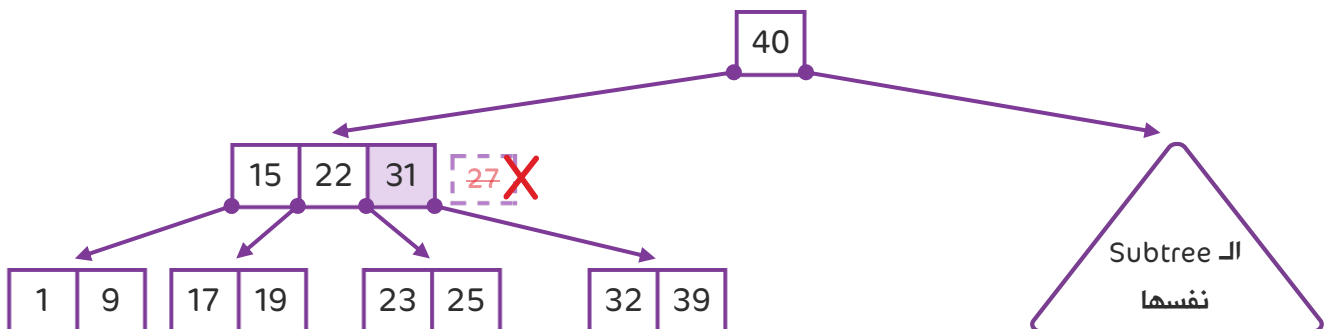


▪ حذف العقدة 27:

العقدة موجودة بصفحة داخلية ليس لها عنصر سابق قابل للحذف البسيط، وإنما لها عنصر تالي قابل للحذف البسيط هو

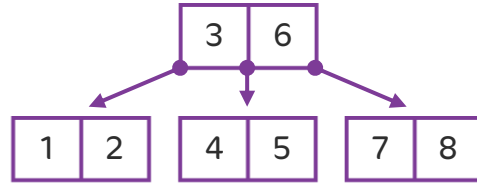
31 ← سنبدل 31 مع 27 ونحذف 27.

← تصبح الشجرة:



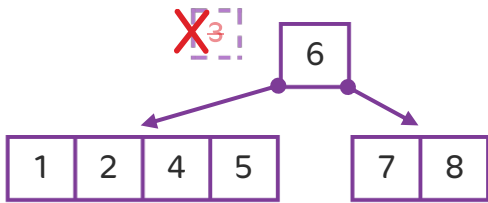
2. الحالة الثانية: الصفحتين التالية والسابقة للعقدة المراد حذفها يملكان t عقدة:

سنقوم بحذف العقدة ودمج ابنيها معاً.



مثال: لدينا الـ B-Tree جانباً حيث $t = 2$:

حذف العقدة 3:

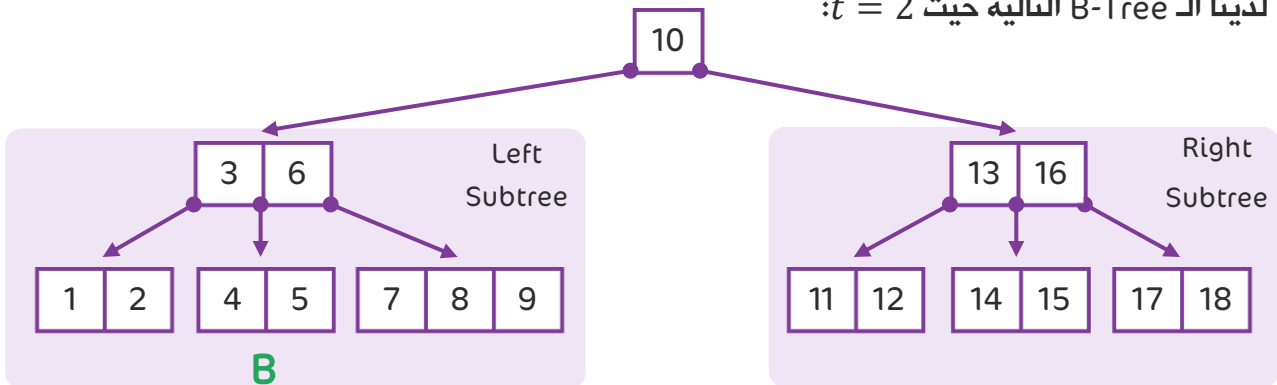


عقدة داخلية وابنيها اليمين واليسار يملكان t عقدة ← نحذفها وندمج ابنيها فتصبح الشجرة:

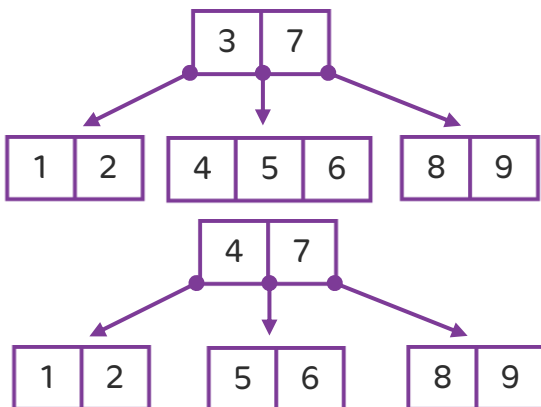
3. الحالة الثالثة: صفحة العقدة التي نريد حذفها لها ابن يملك عقد أكثر من t :

نستعير عقدة من هذه الصفحة مع المحافظة على ترتيب العناصر (الابن اليسار أصغر واليمين أكبر) وذلك من خلال عمليات تبديل وإزاحة.

مثال: لدينا الـ B-Tree التالية حيث $t = 2$:



حذف العقدة 3:



عقدة داخلية أحد أبناء صفحتها يملك أكثر من t عقدة ← نقوم بتغييرات واستعارة من هذه الصفحة ثم نحذف العقدة.

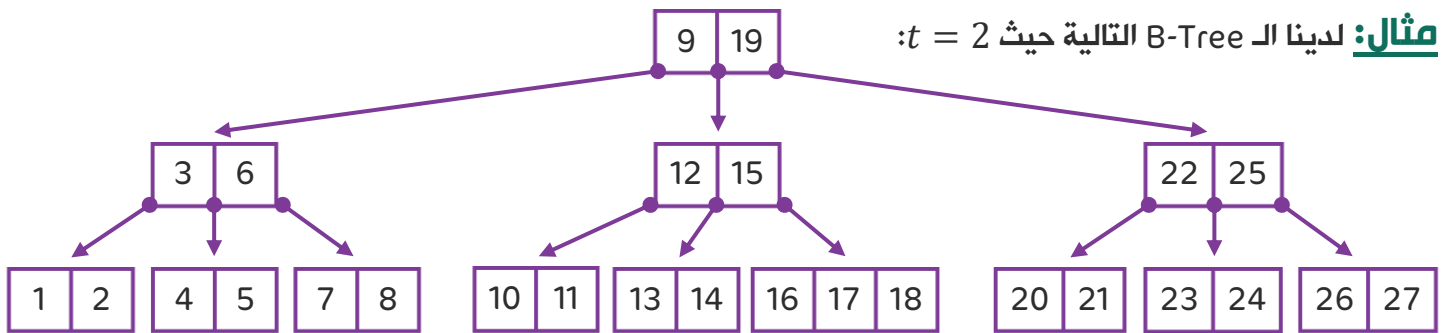
- في الـ Subtree اليسارية لدينا 7 هي العنصر التالي لـ 6 وما سنفعله هو رفع الـ 7 للأب ومن ثم إضافة الـ 6 للصفحة B.

هكذا أصبح بالإمكان الحذف من الصفحة التالية لـ 3 حيث العنصر التالي لـ 3 هو 4 فنبدل ونحذف 3 كما تعلمنا في الحالة الأولى. أما الـ Subtree اليمينية تبقى نفسها.

4. الحالة الرابعة: جميع أبناء وأحفاد صفحة العقدة المراد حذفها تحوي t عقدة فقط:

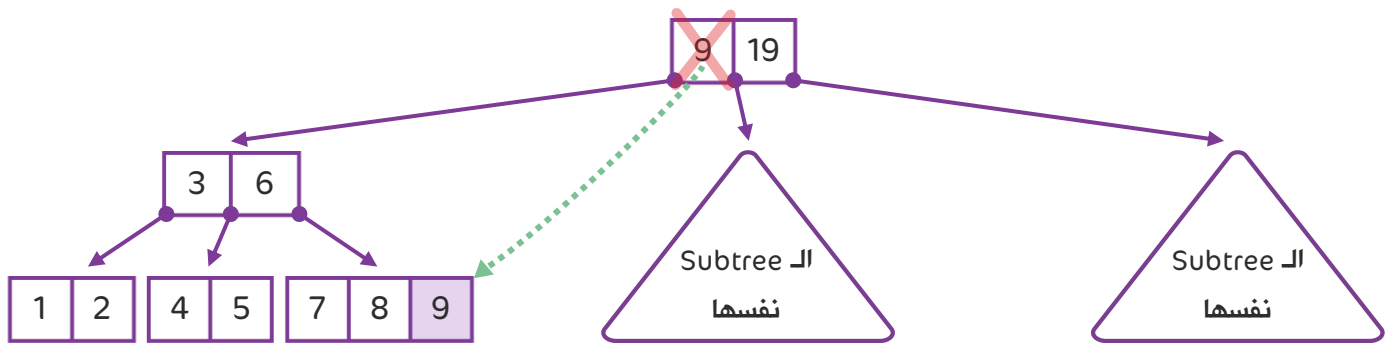
سنستعير عقدة من الأب لنضعها في المكان الذي نحتاجه ضمن الـ Subtree الخاصة بالعقدة المراد حذفها (صفحة العقدة وأبناء الصفحة)، وهكذا يمكننا إعادة الحالة لأحد الحالات الثلاث السابقة، ونعامل العقدة المأخوذة من الأب كأننا حذفنا عقدة فنعالجها هي الأخرى بإحدى الحالات الأربع.

ملاحظة: عند وضع عقدة الأب في المكان المناسب فإنها ستكون ضمن **ورقة** إما في أقصى اليمين أو أقصى اليسار لأن الأب إما أن يكون أكبر من كل العناصر أو أصغر من كل العناصر.



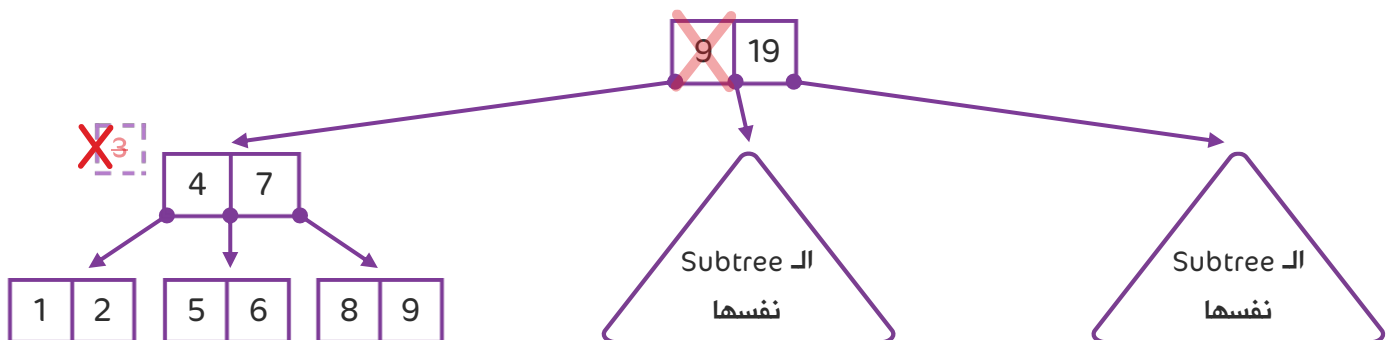
حذف العقدة 3:

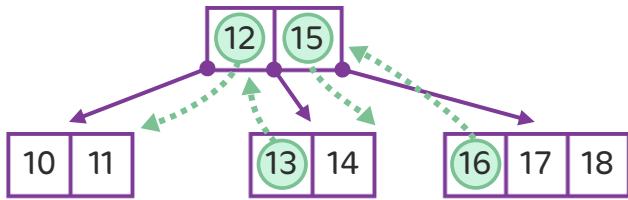
عقدة داخلية ولا نستطيع استبدالها بالعنصر السابق أو التالي وكل أبنائها تحوي t عقدة وكل أبناء صفحاتها تحوي t عقدة
 ← حالة رابعة تتطلب استعارة عقدة من الأب، ولنأخذ 9 ونضعه في مكانه المناسب ضمن صفحة (7,8) أقصى اليمين:



الآن عادت الشجرة لإحدى الحالات الـ 3 السابقة:

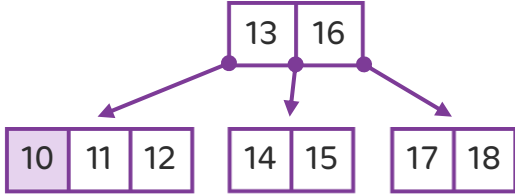
عقدة داخلية ابنيها يملكان t عقدة وأحد أبناء صفحاتها يملك أكثر من t ← الحالة الثالثة:





تبقى معالجة حذف العقدة 9 من الجذر:

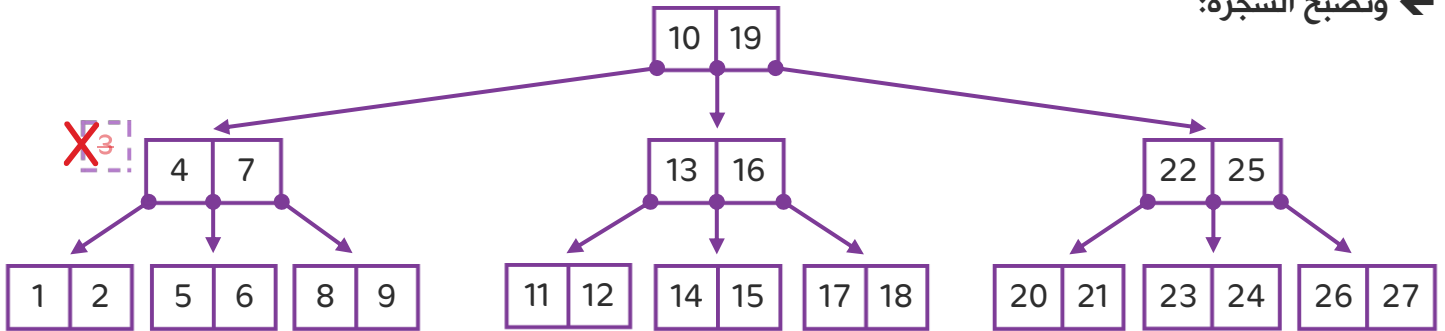
نبحث عن العنصر التالي أو السابق وهما 10 و 8 على التوالي (باعتبار أن العقدة 9 بقيت في مكانها في الجذر خلال البحث عن العنصرين التالي والسابق)، ونلاحظ أن صفحة العنصر التالي تحوي t عقدة ولكن أحد أخواتها تحوي أكثر من t



← الحالة الثالثة لذا سنقوم ببعض التحويلات حتى تصبح الصفحة (10,11) تحوي أكثر من t عقدة وتصبح كالتالي جانباً:

الآن أصبح العنصر التالي لـ 9 وهو 10 قابل للحذف البسيط كما تعلمنا في الحالة الأولى.

← وتصبح الشجرة:



exercises

قم بكتابة برنامج بلغة Java ينشئ شجرة B-Tree تحتوي على مجموعة من الأعداد الصحيحة، ويجب أن يتضمن البرنامج التوابع التالية:

1. تابع لإدراج الأعداد في الشجرة
2. تابع لتقسيم العقدة إذا كانت ممتلئة
3. تابع لإدراج العدد في العقدة المناسبة
4. تابع لطباعة الشجرة
5. تابع للبحث في الشجرة

إدراج الأعداد في الشجرة

```
private static class BTreeNode {
    int[] keys;
    int t;
    BTreeNode[] children;
    int numKeys;
    boolean leaf;
    public BTreeNode(int t, boolean leaf) {
        this.t = t;
        this.leaf = leaf;
        keys = new int[2 * t - 1];
        children = new BTreeNode[2 * t];
    }
}

private BTreeNode root;
private int t;
public BTreeExample(int t) {
    this.t = t;
    root = new BTreeNode(t, true);
}

public void insert(int key) {
    BTreeNode r = root;
    if (r.numKeys == 2 * t - 1) {
        BTreeNode s = new BTreeNode(t,
false);
        root = s;
        s.children[0] = r;
        splitChild(s, 0);
        insertNonFull(s, key);
    } else {
        insertNonFull(r, key);
    }
}
```

تقسيم العقدة إذا كانت ممتلئة

```
private void splitChild (BTreeNode x, int i) {
    BTreeNode y = x.children[i];
    BTreeNode z = new BTreeNode(t, y.leaf);
    x.numKeys++;
    for (int j = t - 1; j >= 0; j--) {
        z.keys[j] = y.keys[j+t];
        y.keys[j+t] = 0;
    }
    if (!y.leaf) {
        for (int j = t; j >= 0; j--) {
            z.children[j] = y.children[j + t];
            y.children[j + t] = null;
        }
    }
    for (int j = x.numKeys - 1; j > i; j--) {
        x.children[j+1] = x.children[j];
    }
    x.children[i+1]=z;
    for (int j = x.numKeys - 2; j >= i; j--){
        x.keys[j+1]=x.keys[j];
    }
    x.keys[i] = y.keys[t - 1];
    y.keys[t - 1] = 0;
    y.numkeys = t - 1;
}
```



إدراج العدد في العقدة المناسبة

```
private static class BTreeNode {
    int[] keys;
    int t;
    BTreeNode[] children;
    int numKeys;
    boolean leaf;
    public BTreeNode(int t, boolean leaf) {
        this.t = t;
        this.leaf = leaf;
        keys = new int[2 * t - 1];
        children = new BTreeNode[2 * t];
    }
}

private BTreeNode root;
private int t;
public BTreeExample(int t) {
    this.t = t;
    root = new BTreeNode(t, true);
}

public void insert(int key) {
    BTreeNode r = root;
    if (r.numKeys == 2 * t - 1) {
        BTreeNode s = new BTreeNode(t, false);
        root = s;
        s.children[0] = r;
        splitChild(s, 0);
        insertNonFull(s, key);
    } else {
        insertNonFull(r, key);
    }
}
```

طباعة الشجرة

```
private static void printBTree(BTreeNode
node) {
    for (int i = 0; i < node.numKeys; i++) {
        System.out.print(node.keys[i] + " ");
    }
    System.out.println();
    if (!node.leaf) {
        for (int i = 0; i <= node.numKeys; i++) {
            printBTree(node.children[i]);
        }
    }
}
```

البحث في الشجرة

```
private boolean search (BTreeNode node,
int key) {
    int i = 0;
    while (i < node.keyCount && key >
node.keys[i]) {
        i++;
    }
    if (i < node.keyCount && node.keys[i] ==
key) {
        return true;
    }
    if (node.isLeaf) {
        return false;
    }
    return search(node.children[i], key);
}
```

The End